Project 5B: Flow Matching from Scratch!

Note: You are not required to use the exact .ipynb notebook +or format (e.g., you can use a .py or modify the .ipynb however you wish). We only need to see the required deliverables / visualizations.

Setup environment

In [1]:
```
# We recommend using these utils.
# https://google.github.io/mediapy/mediapy.html
# https://einops.rocks/
!pip install mediapy einops --quiet
```

```
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0.0/1.6 MB ? eta -:--:--
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.6/1.6 MB 51.0 MB/s eta 0:00:01
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.6/1.6 MB 35.9 MB/s eta 0:00:00
```

In [2]:
```
# Import essential modules. Feel free to add whatever you need.
import matplotlib.pyplot as plt
import torch
from torch import nn
from torch.utils.data import DataLoader
from torchvision.datasets import MNIST
from torchvision.transforms import ToTensor
from tqdm import tqdm
```

Neural Network Resources

In this part, you will build and train a UNet, which is more complex than the MLP you implemented in the NeRF project. We provide all class definitions you may need (but feel free to add or modify them as necessary).

Instead of asking ChatGPT to write everything for you, please consult the following resources when you get stuck — they will help you understand how and why things work under the hood.

- PyTorch Documentation — `Conv2d`, `ConvTranspose2d`, and `AvgPool2d`.
- PyTorch Documentation - `torchvision.datasets.MNIST`, the dataset we gonna use, and `torch.utils.data.DataLoader`, the off-the-shell dataloader we can directly use.
- PyTorch tutorial on how to train a classifier on CIFAR10 dataset. The structure of your training code will be very similar to this one.

Part 1: Training a Single-step Denoising UNet

Part 1.1: Implementing the UNet

Implementing Simple and Composed Ops

In [3]:
```
class Conv(nn.Module):
    def __init__(self, in_channels: int, out_channels: int):
        super().__init__()
        # ===== your code here! ====
        self.net = nn.Sequential(
            nn.Conv2d(in_channels, out_channels, kernel_size=3, stride=1, pa
            nn.BatchNorm2d(out_channels),
            nn.GELU(),
        )
        # ===== end of code ====

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # ===== your code here! ====
        return self.net(x)
        # ===== end of code ====
        raise NotImplementedError()


class DownConv(nn.Module):
    def __init__(self, in_channels: int, out_channels: int):
        super().__init__()
        # ===== your code here! ====
        self.net = nn.Sequential(
            nn.Conv2d(in_channels, out_channels, kernel_size=3, stride=2, pa
            nn.BatchNorm2d(out_channels),
            nn.GELU(),
        )
        # ===== end of code ====

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # ===== your code here! ====
        return self.net(x)
        # ===== end of code ====
        raise NotImplementedError()


class UpConv(nn.Module):
    def __init__(self, in_channels: int, out_channels: int):
        super().__init__()
        # ===== your code here! ====
        self.net = nn.Sequential(
            nn.ConvTranspose2d(in_channels, out_channels, kernel_size=4, str
            nn.BatchNorm2d(out_channels),
            nn.GELU(),
        )
        # ===== end of code ====
```

```
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # ===== your code here! ====
        return self.net(x)
        # ===== end of code ====
        raise NotImplementedError()


class Flatten(nn.Module):
    def __init__(self):
        super().__init__()
        # ===== your code here! ====
        self.net = nn.Sequential(
            nn.AvgPool2d(kernel_size=7),
            nn.GELU(),
        )
        # ===== end of code ====

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # ===== your code here! ====
        return self.net(x)
        # ===== end of code ====
        raise NotImplementedError()


class Unflatten(nn.Module):
    def __init__(self, in_channels: int):
        super().__init__()
        # ===== your code here! ====
        self.net = nn.Sequential(
            nn.ConvTranspose2d(in_channels, in_channels, kernel_size=7, stri
            nn.BatchNorm2d(in_channels),
            nn.GELU(),
        )
        # ===== end of code ====

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # ===== your code here! ====
        return self.net(x)
        # ===== end of code ====
        raise NotImplementedError()


class ConvBlock(nn.Module):
    def __init__(self, in_channels: int, out_channels: int):
        super().__init__()
        # ===== your code here! ====
        self.net = nn.Sequential(
            Conv(in_channels, out_channels),
            Conv(out_channels, out_channels),
        )
        # ===== end of code ====

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # ===== your code here! ====
        return self.net(x)
        # ===== end of code ====
```

```
        raise NotImplementedError()


class DownBlock(nn.Module):
    def __init__(self, in_channels: int, out_channels: int):
        super().__init__()
        # ===== your code here! ====
        self.down = DownConv(in_channels, out_channels)
        self.conv = ConvBlock(out_channels, out_channels)
        # ===== end of code ====

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # ===== your code here! ====
        return self.conv(self.down(x))
        # ===== end of code ====
        raise NotImplementedError()


class UpBlock(nn.Module):
    def __init__(self, in_channels: int, out_channels: int):
        super().__init__()
        # ===== your code here! ====
        self.up = UpConv(in_channels, out_channels)
        self.conv = ConvBlock(out_channels, out_channels)
        # ===== end of code ====

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # ===== your code here! ====
        return self.conv(self.up(x))
        # ===== end of code ====
        raise NotImplementedError()
```

Implementing Unconditional UNet

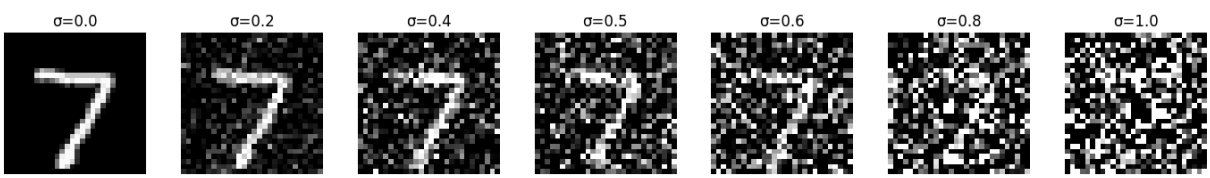
In [5]:

```
class UnconditionalUNet(nn.Module):
    def __init__(
        self,
        in_channels: int,
        num_hiddens: int,
    ):
        super().__init__()
        # ===== your code here! ====
        D = num_hiddens
        self.conv1     = ConvBlock(in_channels, D)
        self.down1     = DownBlock(D, D)
        self.down2     = DownBlock(D, D)
        self.flatten   = Flatten()
        self.unflatten = Unflatten(D)
        self.up1       = UpBlock(2 * D, D)
        self.up2       = UpBlock(2 * D, D)
        self.conv2     = ConvBlock(2 * D, D)
        self.out_conv  = nn.Conv2d(D, in_channels, kernel_size=3, stride=1,
        # ===== end of code ====
```

```
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        assert x.shape[-2:] == (28, 28), "Expect input shape to be (28, 28).
        # ===== your code here! ====
        x1 = self.conv1(x)                    # D,28,28
        x2 = self.down1(x1)                   # D,14,14
        x3 = self.down2(x2)                   # D,7,7
        x4 = self.flatten(x3)                 # D,1,1
        x  = self.unflatten(x4)               # D,7,7
        x  = torch.cat([x, x3], dim=1)        # 2D,7,7
        x  = self.up1(x)                      # D,14,14
        x  = torch.cat([x, x2], dim=1)        # 2D,14,14
        x  = self.up2(x)                      # D,28,28
        x  = torch.cat([x, x1], dim=1)        # 2D,28,28
        x  = self.conv2(x)                    # D,28,28
        return self.out_conv(x)               # C,28,28
        # ===== end of code ====
        raise NotImplementedError()
```

Part 1.2: Using the UNet to Train a Denoiser

In [7]:
```
# Visualize images at different noisy level
# ===== your code here! ====
sigmas = [0.0, 0.2, 0.4, 0.5, 0.6, 0.8, 1.0]
sample_dataset = MNIST(root='./data', train=False, download=True, transform=
sample_img = sample_dataset[0][0]  # shape (1,28,28), in [0,1]

fig, axes = plt.subplots(1, len(sigmas), figsize=(14, 2))
for ax, sigma in zip(axes, sigmas):
    noisy = (sample_img + sigma * torch.randn_like(sample_img)).clamp(0, 1)
    ax.imshow(noisy.squeeze(), cmap='gray')
    ax.set_title(f'σ={sigma}')
    ax.axis('off')
plt.tight_layout()
plt.savefig('noising_process.png', dpi=300, bbox_inches='tight', transparent
# ===== end of code ====
```



Part 1.2.1: Training

For this part, we provide some structure code for training. It is very basic, so feel free to change them or add your code. In later section we won't provide any training or visualization structure code.

In [8]:
```
device = torch.device('cuda')

# Set your hyperparameters
```

```
# ===== your code here! ====
batch_size    = 256
learning_rate = 1e-4
noise_level   = 0.5
hidden_dim    = 128
num_epochs    = 5
# ===== end of code ====
```

In [9]:
```
# Define your datasets and dataloaders
# ===== your code here! ====
train_dataset = MNIST(root='./data', train=True,  download=True, transform=T
test_dataset  = MNIST(root='./data', train=False, download=True, transform=T
train_loader  = DataLoader(train_dataset, batch_size=batch_size, shuffle=Tru
test_loader   = DataLoader(test_dataset,  batch_size=batch_size, shuffle=Fal
# ===== end of code ====
```

In [65]:
```
# Define your model, optimizer, and loss
# ===== your code here! ====
model     = UnconditionalUNet(in_channels=1, num_hiddens=hidden_dim).to(devi
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)
criterion = nn.MSELoss()
# ===== end of code ====
```

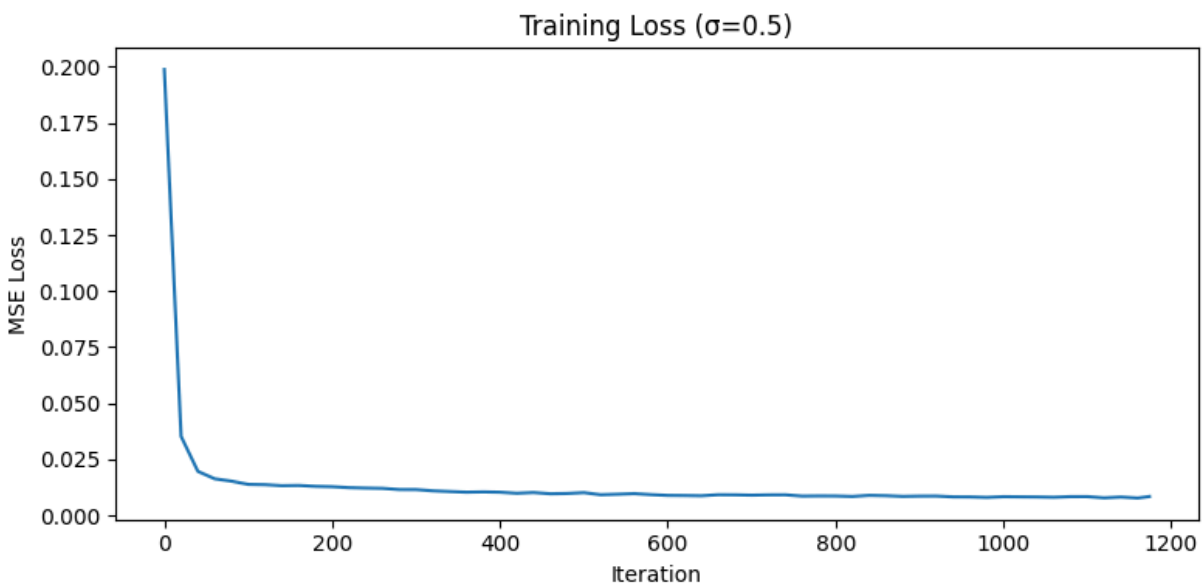
In [66]:
```
# The training loops

# Fixed test batch
test_iter = iter(test_loader)
sample_images, _ = next(test_iter)
sample_images = sample_images.to(device)

saved = {}

train_losses = []
for epoch in range(num_epochs):
    for i, (images, _) in enumerate(tqdm(train_loader)):

        # ===== your code here! ====

        images = images.to(device)
        noisy  = images + noise_level * torch.randn_like(images)
        outputs = model(noisy)

        # ===== end of code ====

        loss = criterion(outputs, images)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        train_losses.append(loss.item())
    if epoch == 0 or epoch == 4:
        model.eval()
        with torch.no_grad():
            noisy_imgs = sample_images + noise_level * torch.randn_like(samp
            outputs = model(noisy_imgs)
```

```
        saved[epoch] = {
            "input": sample_images.detach().cpu(),
            "noisy": noisy_imgs.detach().cpu(),
            "output": outputs.detach().cpu()
        }

        model.train()
```

```
100%|██████████| 235/235 [01:24<00:00,  2.77it/s]
100%|██████████| 235/235 [01:24<00:00,  2.78it/s]
100%|██████████| 235/235 [01:24<00:00,  2.78it/s]
100%|██████████| 235/235 [01:24<00:00,  2.78it/s]
100%|██████████| 235/235 [01:24<00:00,  2.79it/s]
```

In [60]:

```
# Visualize your training curve
# ===== your code here! ====
torch.save(model.state_dict(), 'model_weights1.pth')
sampled_losses = train_losses[::20] + [train_losses[-1]]
iterations = list(range(0, len(train_losses), 20)) + [ len(train_losses)-1]
plt.figure(figsize=(8, 4))
plt.plot(iterations, sampled_losses)
plt.xlabel('Iteration')
plt.ylabel('MSE Loss')
plt.title(f'Training Loss (σ={noise_level})')
plt.tight_layout()
plt.savefig('training_curve1.png', dpi=300, bbox_inches='tight', transparent
# ===== end of code ====
```



In [56]:

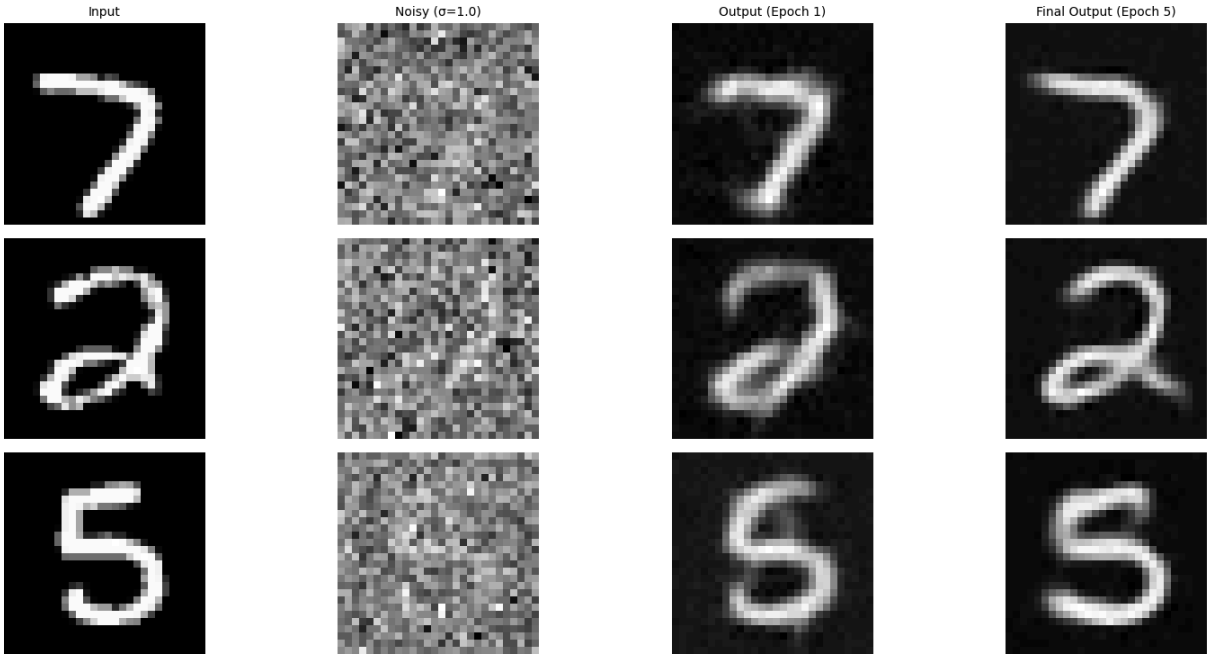
```
len(saved[4]["noisy"])
```

Out[56]: 256

In [70]:

```
num_images = 3

inputs = saved[0]["input"]
noisy = saved[0]["noisy"]
```

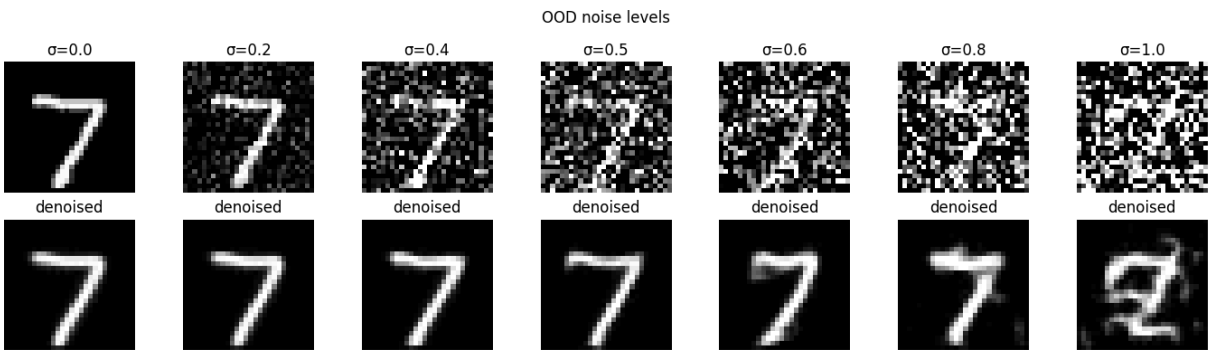
```
out1 = saved[0]["output"]
out5 = saved[4]["output"]

fig, axes = plt.subplots(num_images, 4, figsize=(16, 2.5 * num_images))

titles = ["Input", f"Noisy (σ={noise_level})", "Output (Epoch 1)", "Final Ou
for j in range(4):
    axes[0, j].set_title(titles[j], fontsize=10)

for k, i in enumerate([17, 106, 165]):
    axes[k, 0].imshow(inputs[i].squeeze(), cmap='gray')
    axes[k, 0].axis('off')

    axes[k, 1].imshow(noisy[i].squeeze(), cmap='gray')
    axes[k, 1].axis('off')

    axes[k, 2].imshow(out1[i].squeeze(), cmap='gray')
    axes[k, 2].axis('off')

    axes[k, 3].imshow(out5[i].squeeze(), cmap='gray')
    axes[k, 3].axis('off')

plt.tight_layout()
plt.savefig('sample_results1.png', dpi=300, bbox_inches='tight', transparent
```



Part 1.2.2: Out-of-Distribution Testing

In [61]:
```
# Visualize OOD testing
# ===== your code here! ====
sigmas = [0.0, 0.2, 0.4, 0.5, 0.6, 0.8, 1.0]
model.eval()
sample_img, _ = next(iter(test_loader))
sample_img = sample_img[:1].to(device)   # keep 1 image fixed
```

```
fig, axes = plt.subplots(2, len(sigmas), figsize=(14, 4))
for col, sigma in enumerate(sigmas):
    noisy   = (sample_img + sigma * torch.randn_like(sample_img))
    with torch.no_grad():
        denoised = model(noisy)
    axes[0, col].imshow(noisy.squeeze().cpu().clamp(0,1), cmap='gray')
    axes[0, col].set_title(f'σ={sigma}')
    axes[0, col].axis('off')
    axes[1, col].imshow(denoised.squeeze().cpu().clamp(0,1), cmap='gray')
    axes[1, col].set_title('denoised')
    axes[1, col].axis('off')
plt.suptitle('OOD noise levels')
plt.tight_layout()
plt.savefig('ood.png', dpi=300, bbox_inches='tight', transparent=True)
# ===== end of code ====
```

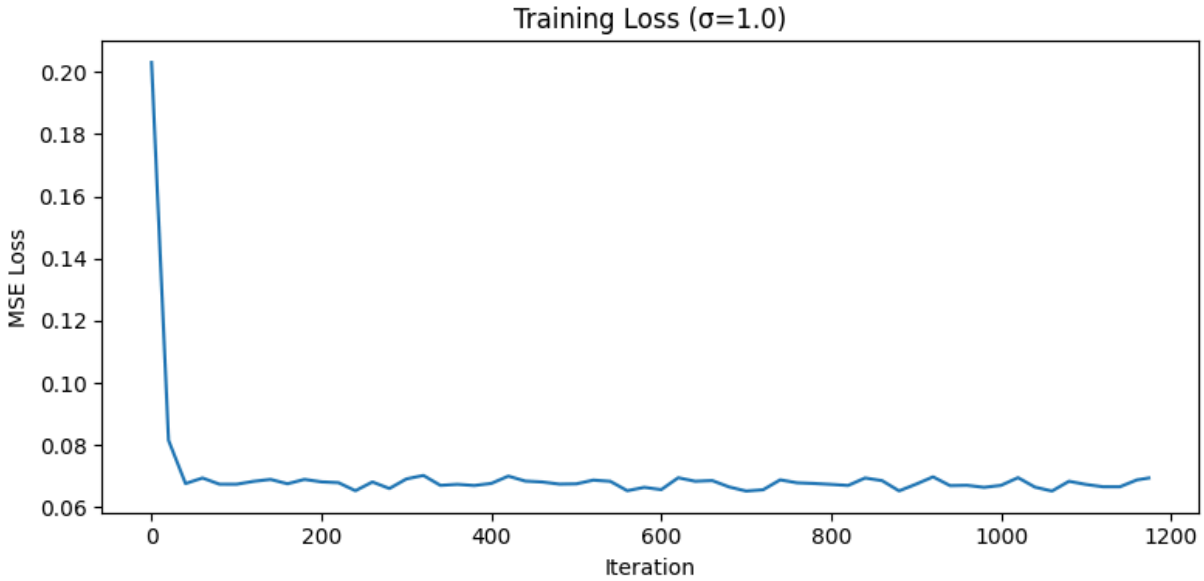


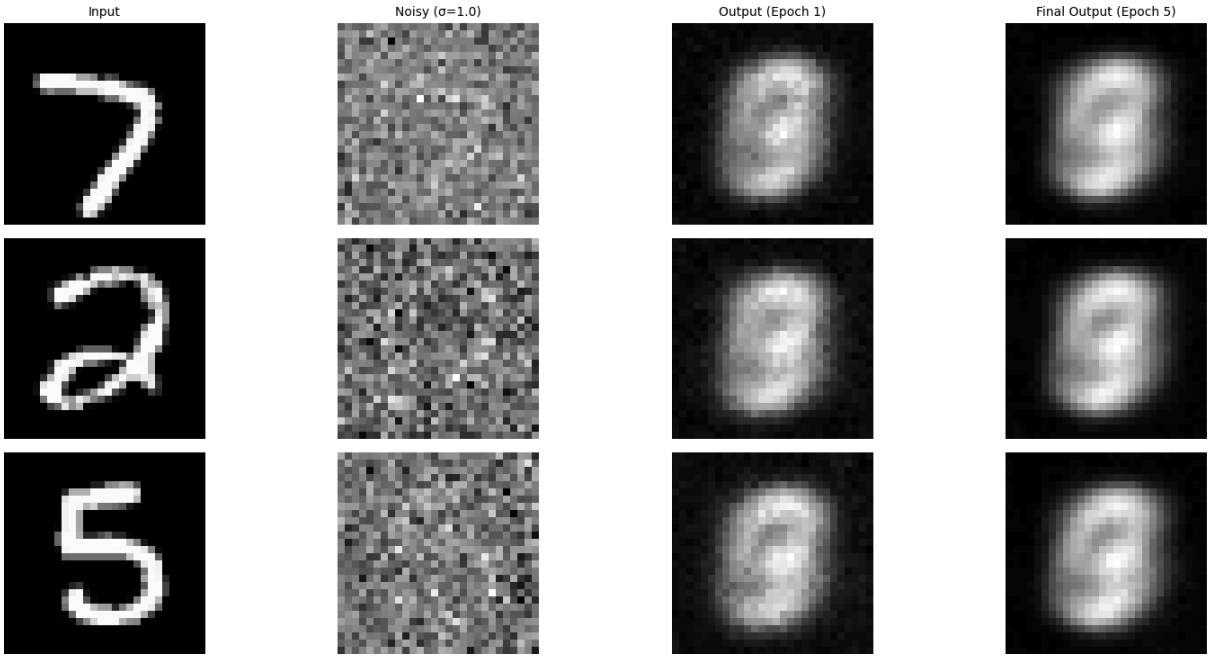
Part 1.2.3 Denoising Pure Noise

In [71]:
```
# Feel free to use code from part 1.2.1
# as they should be very similar
# ===== your code here! ====
# Train a new model to denoise pure Gaussian noise (sigma=1 always)
noise_level = 1.0
pure_noise_model     = UnconditionalUNet(in_channels=1, num_hiddens=hidden_c
pure_noise_optimizer = torch.optim.Adam(pure_noise_model.parameters(), lr=le
pure_noise_criterion = nn.MSELoss()

# Fixed test batch
test_iter = iter(test_loader)
sample_images, _ = next(test_iter)
sample_images = sample_images.to(device)

pure_noise_losses = []
saved_pure_noise = {}
for epoch in range(num_epochs):
    pure_noise_model.train()
    for i, (imgs, _) in enumerate(tqdm(train_loader)):
        imgs  = imgs.to(device)
        noise = torch.randn_like(imgs)          # pure Gaussian noise as inp
        out   = pure_noise_model(noise)
        loss  = pure_noise_criterion(out, imgs)
        pure_noise_optimizer.zero_grad()
        loss.backward()
```

```
        pure_noise_optimizer.step()
        pure_noise_losses.append(loss.item())

    # Visualize after epoch 1 and epoch 5
    if epoch == 0 or epoch == 4:
        pure_noise_model.eval()
        with torch.no_grad():
            noisy_imgs = sample_images + noise_level * torch.randn_like(samp
            outputs = pure_noise_model(noisy_imgs)

        saved_pure_noise[epoch] = {
            "input": sample_images.detach().cpu(),
            "noisy": noisy_imgs.detach().cpu(),
            "output": outputs.detach().cpu()
        }

# Visualize Training Curve
sampled_losses = pure_noise_losses[::20] + [pure_noise_losses[-1]]
iterations = list(range(0, len(train_losses), 20)) + [ len(train_losses)-1]
plt.figure(figsize=(8, 4))
plt.plot(iterations, sampled_losses)
plt.xlabel('Iteration')
plt.ylabel('MSE Loss')
plt.title(f'Training Loss (σ={noise_level})')
plt.tight_layout()
plt.savefig('training_curve_pure_noise.png', dpi=300, bbox_inches='tight', t

# Sample Results
num_images = 3
inputs = saved_pure_noise[0]["input"]
noisy = saved_pure_noise[0]["noisy"]
out1 = saved_pure_noise[0]["output"]
out5 = saved_pure_noise[4]["output"]
fig, axes = plt.subplots(num_images, 4, figsize=(16, 2.5 * num_images))
titles = ["Input", f"Noisy (σ={noise_level})", "Output (Epoch 1)", "Final Ou
for j in range(4):
    axes[0, j].set_title(titles[j], fontsize=10)
for k, i in enumerate([17, 106, 165]):
    axes[k, 0].imshow(inputs[i].squeeze(), cmap='gray')
    axes[k, 0].axis('off')

    axes[k, 1].imshow(noisy[i].squeeze(), cmap='gray')
    axes[k, 1].axis('off')

    axes[k, 2].imshow(out1[i].squeeze(), cmap='gray')
    axes[k, 2].axis('off')

    axes[k, 3].imshow(out5[i].squeeze(), cmap='gray')
    axes[k, 3].axis('off')
plt.tight_layout()
plt.savefig('sample_results_pure_noise.png', dpi=300, bbox_inches='tight', t
# ===== end of code ====
```

```
100%|██████████| 235/235 [01:20<00:00,  2.91it/s]
100%|██████████| 235/235 [01:21<00:00,  2.87it/s]
100%|██████████| 235/235 [01:23<00:00,  2.82it/s]
100%|██████████| 235/235 [01:24<00:00,  2.79it/s]
100%|██████████| 235/235 [01:24<00:00,  2.78it/s]
```





Part 2: Flow Matching

Part 2.1: Implementing a Time-conditioned UNet

In [72]:
```
class FCBlock(nn.Module):
    def __init__(self, in_channels: int, out_channels: int):
```

```python
        super().__init__()
        # ===== your code here! ====
        self.net = nn.Sequential(
            nn.Linear(in_channels, out_channels),
            nn.GELU(),
            nn.Linear(out_channels, out_channels),
        )
        # ===== end of code ====

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # ===== your code here! ====
        return self.net(x)
        # ===== end of code ====
        raise NotImplementedError()


class TimeConditionalUNet(nn.Module):
    def __init__(
        self,
        in_channels: int,
        num_hiddens: int,
    ):
        super().__init__()
        # ===== your code here! ====
        D = num_hiddens
        self.conv1     = ConvBlock(in_channels, D)
        self.down1     = DownBlock(D, D)
        self.down2     = DownBlock(D, D)
        self.flatten   = Flatten()
        self.unflatten = Unflatten(D)
        self.up1       = UpBlock(2 * D, D)
        self.up2       = UpBlock(2 * D, D)
        self.conv2     = ConvBlock(2 * D, D)
        self.out_conv  = nn.Conv2d(D, in_channels, kernel_size=3, stride=1,
        # Time conditioning FCBlocks — inject at unflatten and up1 levels
        # D channels each, scalar t input (size 1)
        self.fc1_t = FCBlock(1, D)   # modulates unflatten (D,7,7)
        self.fc2_t = FCBlock(1, D)   # modulates up1 output (D,14,14)
        # ===== end of code ====

    def forward(
        self,
        x: torch.Tensor,
        t: torch.Tensor,
    ) -> torch.Tensor:
        """
        Args:
            x: (N, C, H, W) input tensor.
            t: (N,) normalized time tensor.

        Returns:
            (N, C, H, W) output tensor.
        """
        assert x.shape[-2:] == (28, 28), "Expect input shape to be (28, 28).
        # ===== your code here! ====
        t_in = t.view(-1, 1).float()
```

```
        t1 = self.fc1_t(t_in).view(t_in.shape[0], -1, 1, 1)  # (N,D,1,1)
        t2 = self.fc2_t(t_in).view(t_in.shape[0], -1, 1, 1)  # (N,D,1,1)

        x1 = self.conv1(x)
        x2 = self.down1(x1)
        x3 = self.down2(x2)
        x4 = self.flatten(x3)

        unflat = self.unflatten(x4) * t1
        x      = torch.cat([unflat, x3], dim=1)

        up1 = self.up1(x) * t2
        x   = torch.cat([up1, x2], dim=1)

        x = self.up2(x)
        x = torch.cat([x, x1], dim=1)
        x = self.conv2(x)
        return self.out_conv(x)
        # ===== end of code ====
        raise NotImplementedError()
```

Implementing the Forward and Reverse Process for Time-conditioned Denoising

In [73]:

```
def time_fm_forward(
    unet: TimeConditionalUNet,
    x_1: torch.Tensor,
    num_ts: int,
) -> torch.Tensor:
    """Algorithm 1

    Args:
        unet: TimeConditionalUNet
        x_1: (N, C, H, W) input tensor.
        num_ts: int, number of timesteps.
    Returns:
        (,) loss.
    """
    unet.train()
    # ===== your code here! ====
    unet.train()
    # ===== your code here! ====
    N = x_1.shape[0]
    t = torch.rand(N, device=x_1.device)
    x_0 = torch.randn_like(x_1)
    x_t = (1 - t[:, None, None, None]) * x_0 + t[:, None, None, None] * x_1
    target = x_1 - x_0
    pred   = unet(x_t, t)
    return nn.functional.mse_loss(pred, target)
    # ===== end of code ====
    raise NotImplementedError()
    # ===== end of code ====
    raise NotImplementedError()
```

In [74]:
```
@torch.inference_mode()
def time_fm_sample(
    unet: TimeConditionalUNet,
    img_wh: tuple[int, int],
    num_ts: int,
    seed: int = 0,
) -> torch.Tensor:
    """Algorithm 2

    Args:
        unet: TimeConditionalUNet
        img_wh: (H, W) output image width and height.
        num_ts: int, number of timesteps.
        seed: int, random seed.

    Returns:
        (N, C, H, W) final sample.
    """
    unet.eval()
    # ===== your code here! ====
    unet.eval()
    # ===== your code here! ====
    torch.manual_seed(seed)
    H, W   = img_wh
    N      = 16                                        # number of samples
    device = next(unet.parameters()).device
    x_t    = torch.randn(N, 1, H, W, device=device) # start from pure noise

    step_size = 1.0 / num_ts
    for i in range(num_ts):
        t_val = i / num_ts
        t     = torch.full((N,), t_val, device=device)
        u     = unet(x_t, t)
        x_t   = x_t + step_size * u

    return x_t
    # ===== end of code ====
    raise NotImplementedError()
```

In [75]:
```
class TimeConditionalFM(nn.Module):
    def __init__(
        self,
        unet: TimeConditionalUNet,
        num_ts: int = 50,
        img_hw: tuple[int, int] = (28, 28),
    ):
        super().__init__()

        self.unet = unet
        self.num_ts = num_ts
        self.img_hw = img_hw


    def forward(self, x: torch.Tensor) -> torch.Tensor:
        """
```

```
        Args:
            x: (N, C, H, W) input tensor.

        Returns:
            (,) diffusion loss.
        """
        return time_fm_forward(
            self.unet, x, self.num_ts
        )


    @torch.inference_mode()
    def sample(
        self,
        img_wh: tuple[int, int],
        seed: int = 0,
    ):
        return time_fm_sample(
            self.unet, img_wh, self.num_ts, seed
        )
```

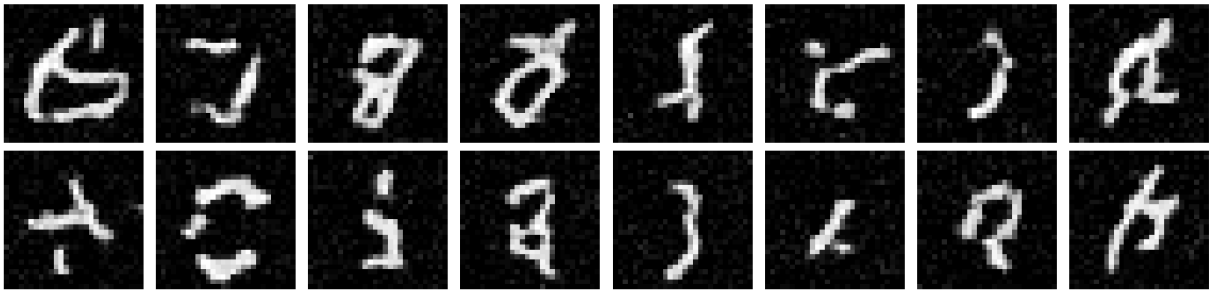
Part 2.2: Training the Time-conditioned UNet

In [88]:
```
# Feel free to use code from part 1.2.1
# as they should be very similar
# ===== your code here! ====
fm_hidden_dim  = 64
fm_batch_size  = 64
fm_lr          = 1e-2
fm_num_epochs  = 10

fm_train_dataset = MNIST(root='./data', train=True,  download=True, transfor
fm_train_loader  = DataLoader(fm_train_dataset, batch_size=fm_batch_size, sh

fm_unet  = TimeConditionalUNet(in_channels=1, num_hiddens=fm_hidden_dim).to(
fm_model = TimeConditionalFM(fm_unet, num_ts=50).to(device)
fm_optim = torch.optim.Adam(fm_model.parameters(), lr=fm_lr)
fm_sched = torch.optim.lr_scheduler.ExponentialLR(
    fm_optim, gamma=0.1 ** (1.0 / fm_num_epochs)
)

fm_losses = []
for epoch in range(fm_num_epochs):
    fm_model.train()
    for imgs, _ in tqdm(fm_train_loader):
        imgs = imgs.to(device)
        loss = fm_model(imgs)
        fm_optim.zero_grad()
        loss.backward()
        fm_optim.step()
        fm_losses.append(loss.item())
    fm_sched.step()
    if epoch in [0, 4, 9]:
        fm_model.eval()
```

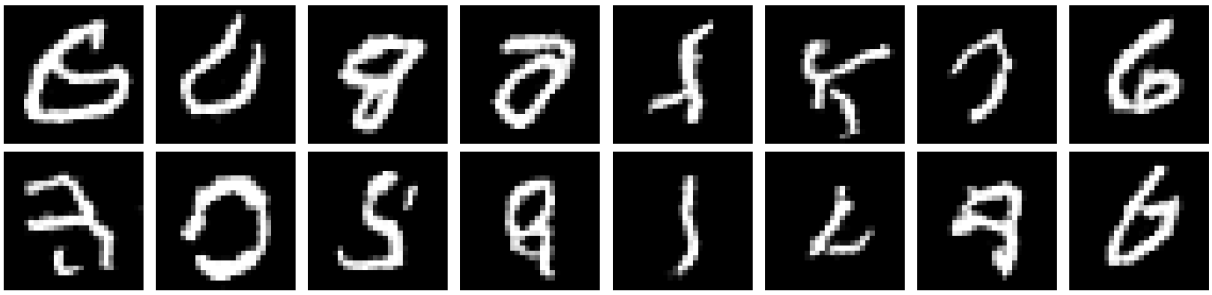
```
        samples = fm_model.sample(img_wh=(28, 28), seed=42).cpu().clamp(0, 1
        fig, axes = plt.subplots(2, 8, figsize=(14, 4))
        for idx, ax in enumerate(axes.flatten()):
            ax.imshow(samples[idx].squeeze(), cmap='gray')
            ax.axis('off')
        plt.subplots_adjust(wspace=0, hspace=0)
        plt.margins(0, 0)

        # remove outer padding completely
        for ax_row in axes:
            for ax in ax_row:
                ax.set_position(ax.get_position())  # ensures tight packing
        plt.suptitle(f'Time-conditioned FM samples — Epoch {epoch+1}')
        plt.tight_layout()
        plt.savefig(f'time_conditioned_sample{epoch+1}.png', dpi=300, bbox_i

sampled_losses = fm_losses[::20] + [fm_losses[-1]]
iterations = list(range(0, len(fm_losses), 20)) + [len(fm_losses)-1]
plt.figure(figsize=(8, 4))
plt.plot(iterations, sampled_losses)
plt.xlabel('Iteration')
plt.ylabel('MSE Loss')
plt.title('Time-conditioned Flow Matching Training Loss')
plt.tight_layout()
plt.savefig('training_curve_fm.png', dpi=300, bbox_inches='tight', transpare
# ===== end of code ====
```

```
100%|██████████| 938/938 [00:42<00:00, 22.19it/s]
100%|██████████| 938/938 [00:40<00:00, 22.92it/s]
100%|██████████| 938/938 [00:42<00:00, 21.86it/s]
100%|██████████| 938/938 [00:40<00:00, 22.90it/s]
100%|██████████| 938/938 [00:41<00:00, 22.34it/s]
100%|██████████| 938/938 [00:41<00:00, 22.44it/s]
100%|██████████| 938/938 [00:39<00:00, 23.46it/s]
100%|██████████| 938/938 [00:40<00:00, 23.10it/s]
100%|██████████| 938/938 [00:39<00:00, 23.52it/s]
100%|██████████| 938/938 [00:40<00:00, 23.31it/s]
```

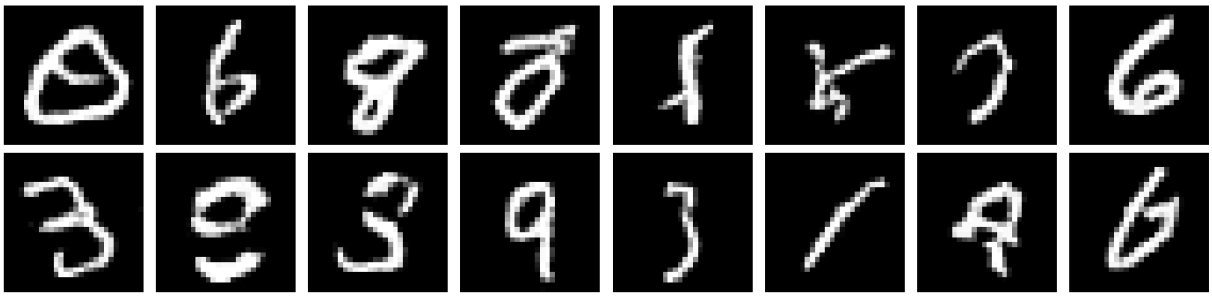
Time-conditioned FM samples — Epoch 1

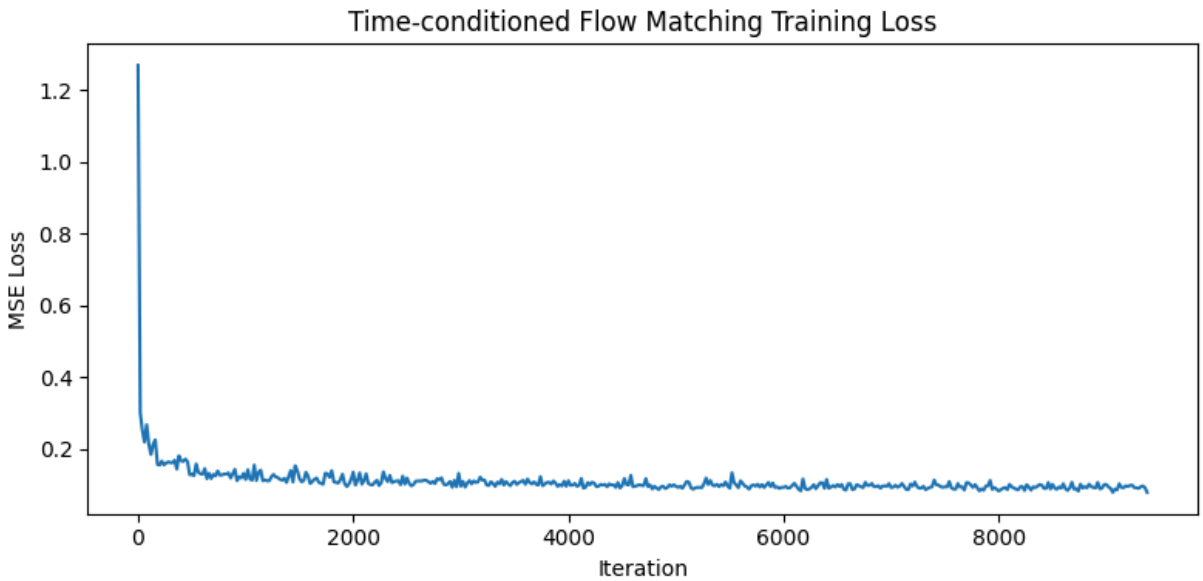


Time-conditioned FM samples — Epoch 5



Time-conditioned FM samples — Epoch 10





Part 2.3: Sampling from the Time-conditioned UNet

In [79]:
```
# Sampling from the UNet
# ===== your code here! ====
# DONE IN THE CELL ABOVE
# ===== end of code ====
```

Part 2.4: Implementing a Class-conditioned UNet

In [80]:
```
class ClassConditionalUNet(nn.Module):
    def __init__(
        self,
        in_channels: int,
        num_classes: int,
        num_hiddens: int,
    ):
        super().__init__()
        # ===== your code here! ====
        D = num_hiddens
        self.conv1     = ConvBlock(in_channels, D)
        self.down1     = DownBlock(D, D)
        self.down2     = DownBlock(D, D)
        self.flatten   = Flatten()
        self.unflatten = Unflatten(D)
        self.up1       = UpBlock(2 * D, D)
        self.up2       = UpBlock(2 * D, D)
        self.conv2     = ConvBlock(2 * D, D)
        self.out_conv  = nn.Conv2d(D, in_channels, kernel_size=3, stride=1,
        self.fc1_t = FCBlock(1, D)
        self.fc1_c = FCBlock(num_classes, D)
        self.fc2_t = FCBlock(1, D)
        self.fc2_c = FCBlock(num_classes, D)
        self.num_classes = num_classes
        # ===== end of code ====

    def forward(
        self,
        x: torch.Tensor,
        c: torch.Tensor,
        t: torch.Tensor,
        mask: torch.Tensor | None = None,
    ) -> torch.Tensor:
        """
        Args:
            x: (N, C, H, W) input tensor.
            c: (N,) int64 condition tensor.
            t: (N,) normalized time tensor.
            mask: (N,) mask tensor. If not None, mask out condition when mas

        Returns:
            (N, C, H, W) output tensor.
        """
        assert x.shape[-2:] == (28, 28), "Expect input shape to be (28, 28).
        # ===== your code here! ====
        N = x.shape[0]
        t_in   = t.view(-1, 1).float()

        # One-hot encode class labels
        c_onehot = torch.zeros(N, self.num_classes, device=x.device)
        c_onehot.scatter_(1, c.view(-1, 1), 1.0)
        if mask is not None:
            c_onehot = c_onehot * mask.view(-1, 1).float()

        t1 = self.fc1_t(t_in).view(N, -1, 1, 1)
```

```
        c1 = self.fc1_c(c_onehot).view(N, -1, 1, 1)
        t2 = self.fc2_t(t_in).view(N, -1, 1, 1)
        c2 = self.fc2_c(c_onehot).view(N, -1, 1, 1)

        x1 = self.conv1(x)
        x2 = self.down1(x1)
        x3 = self.down2(x2)
        x4 = self.flatten(x3)

        unflat = self.unflatten(x4)
        unflat = c1 * unflat + t1
        x = torch.cat([unflat, x3], dim=1)

        up1 = self.up1(x)
        up1 = c2 * up1 + t2
        x = torch.cat([up1, x2], dim=1)

        x = self.up2(x)
        x = torch.cat([x, x1], dim=1)
        x = self.conv2(x)
        return self.out_conv(x)
        # ===== end of code ====
        raise NotImplementedError()
```

In [81]:

```
def class_fm_forward(
    unet: ClassConditionalUNet,
    x_1: torch.Tensor,
    c: torch.Tensor,
    p_uncond: float,
    num_ts: int,
) -> torch.Tensor:
    """Algorithm 3

    Args:
        unet: ClassConditionalUNet
        x_1: (N, C, H, W) input tensor.
        c: (N,) int64 condition tensor.
        p_uncond: float, probability of unconditioning the condition.
        num_ts: int, number of timesteps.

    Returns:
        (,) loss.
    """
    unet.train()
    # ===== your code here! ====
    N      = x_1.shape[0]
    t      = torch.rand(N, device=x_1.device)
    x_0    = torch.randn_like(x_1)
    x_t    = (1 - t[:, None, None, None]) * x_0 + t[:, None, None, None] * x
    target = x_1 - x_0

    # Dropout: zero out class conditioning with probability p_uncond
    mask = (torch.rand(N, device=x_1.device) >= p_uncond).long()
    pred = unet(x_t, c, t, mask=mask)
    return nn.functional.mse_loss(pred, target)
```

```
    # ===== end of code ====
    raise NotImplementedError()
```

In [82]:
```
@torch.inference_mode()
def class_fm_sample(
    unet: ClassConditionalUNet,
    c: torch.Tensor,
    img_wh: tuple[int, int],
    num_ts: int,
    guidance_scale: float = 5.0,
    seed: int = 0,
) -> torch.Tensor:
    """Algorithm 4

    Args:
        unet: ClassConditionalUNet
        c: (N,) int64 condition tensor. Only for class-conditional
        img_wh: (H, W) output image width and height.
        num_ts: int, number of timesteps.
        guidance_scale: float, CFG scale.
        seed: int, random seed.

    Returns:
        (N, C, H, W) final sample.
        (N, T_animation, C, H, W) caches.
    """
    unet.eval()
    # ===== your code here! ====
    torch.manual_seed(seed)
    H, W   = img_wh
    N      = c.shape[0]
    device = next(unet.parameters()).device
    x_t    = torch.randn(N, 1, H, W, device=device)
    c      = c.to(device)
    zeros  = torch.zeros(N, device=device).long()   # null class mask → all
    mask_on  = torch.ones(N, device=device).long()
    mask_off = torch.zeros(N, device=device).long()

    step_size = 1.0 / num_ts
    for i in range(num_ts):
        t_val = i / num_ts
        t     = torch.full((N,), t_val, device=device)

        u_uncond = unet(x_t, c, t, mask=mask_off)   # c ignored (mask=0)
        u_cond   = unet(x_t, c, t, mask=mask_on)    # c used    (mask=1)
        u        = u_uncond + guidance_scale * (u_cond - u_uncond)
        x_t      = x_t + step_size * u

    return x_t, None   # (samples, caches) — caches omitted
    # ===== end of code ====
    raise NotImplementedError()
```

In [83]:
```
class ClassConditionalFM(nn.Module):
    def __init__(
        self,
```

```
        unet: ClassConditionalUNet,
        num_ts: int = 300,
        p_uncond: float = 0.1,
    ):
        super().__init__()
        self.unet = unet
        self.num_ts = num_ts
        self.p_uncond = p_uncond

    def forward(self, x: torch.Tensor, c: torch.Tensor) -> torch.Tensor:
        """
        Args:
            x: (N, C, H, W) input tensor.
            c: (N,) int64 condition tensor.

        Returns:
            (,) loss.
        """
        return class_fm_forward(
            self.unet, x, c, self.p_uncond, self.num_ts
        )

    @torch.inference_mode()
    def sample(
        self,
        c: torch.Tensor,
        img_wh: tuple[int, int],
        guidance_scale: float = 5.0,
        seed: int = 0,
    ):
        return class_fm_sample(
            self.unet, c, img_wh, self.num_ts, guidance_scale, seed
        )
```

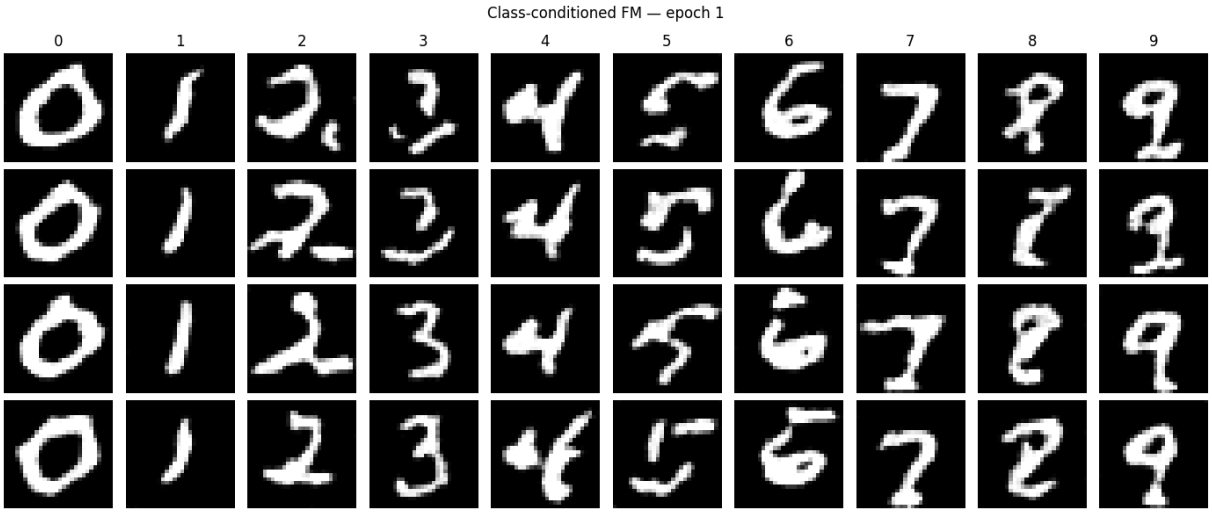
Part 2.5 Training the Class-conditioned UNet

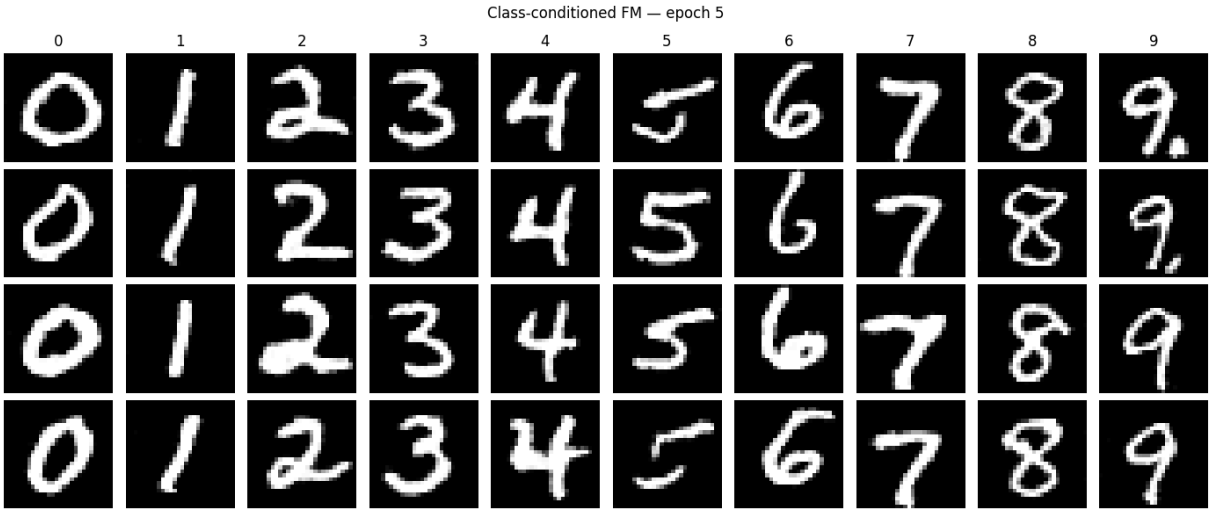
In [87]:

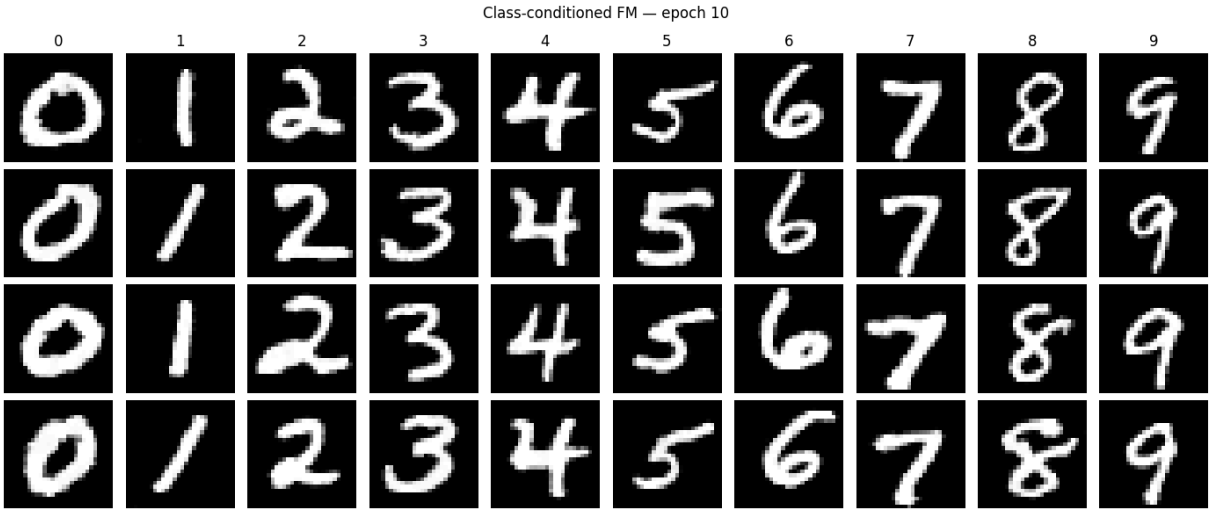
```
# Feel free to use code from part 1.2.1
# as they should be very similar
# ===== your code here! ====
cc_hidden_dim  = 64
cc_batch_size  = 64
cc_lr          = 1e-2
cc_num_epochs  = 10

cc_train_dataset = MNIST(root='./data', train=True,  download=True, transfor
cc_train_loader  = DataLoader(cc_train_dataset, batch_size=cc_batch_size, sh

cc_unet  = ClassConditionalUNet(in_channels=1, num_classes=10, num_hiddens=c
cc_model = ClassConditionalFM(cc_unet, num_ts=50, p_uncond=0.1).to(device)
cc_optim = torch.optim.Adam(cc_model.parameters(), lr=cc_lr)
cc_sched = torch.optim.lr_scheduler.ExponentialLR(
    cc_optim, gamma=0.1 ** (1.0 / cc_num_epochs)
```

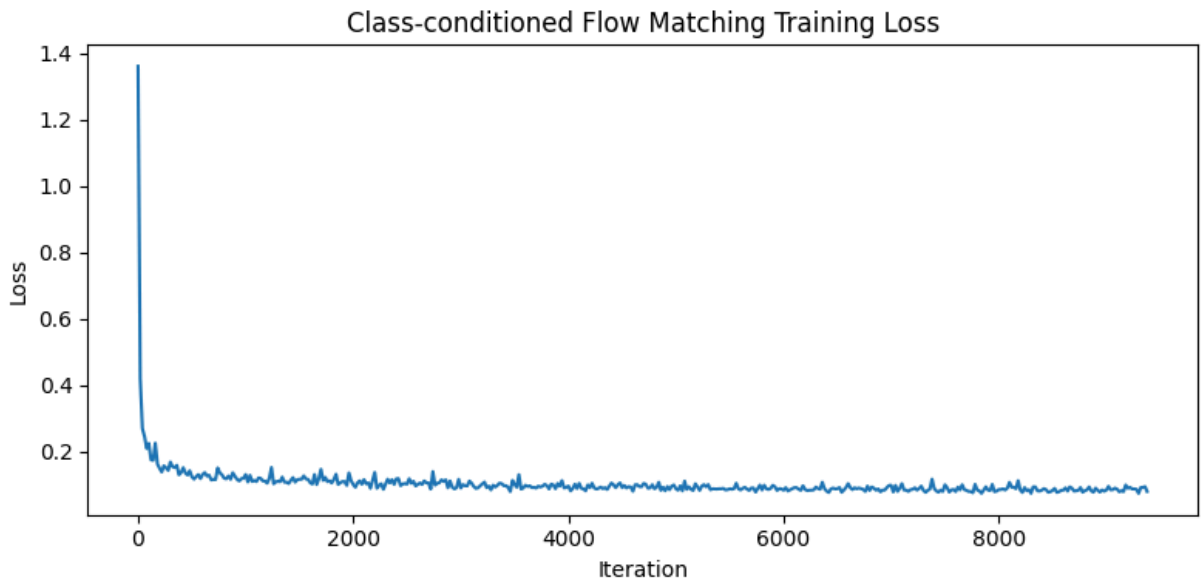
```
)

cc_losses = []
for epoch in range(cc_num_epochs):
    cc_model.train()
    for imgs, labels in tqdm(cc_train_loader):
        imgs   = imgs.to(device)
        labels = labels.to(device)
        loss   = cc_model(imgs, labels)
        cc_optim.zero_grad()
        loss.backward()
        cc_optim.step()
        cc_losses.append(loss.item())
    cc_sched.step()

    if epoch + 1 in (1, 5, cc_num_epochs):
        cc_model.eval()
        # Generate 4 samples per digit (4×10 = 40 images)
        c_all = torch.arange(10, device=device).repeat_interleave(4)
        samples, _ = cc_model.sample(c=c_all, img_wh=(28, 28), seed=42)
        samples = samples.cpu().clamp(0, 1)
        fig, axes = plt.subplots(4, 10, figsize=(14, 6))
        for digit in range(10):
            for row in range(4):
                ax = axes[row, digit]
                ax.imshow(samples[digit * 4 + row].squeeze(), cmap='gray')
                ax.axis('off')
                if row == 0:
                    ax.set_title(str(digit))
        plt.subplots_adjust(wspace=0, hspace=0)
        plt.margins(0, 0)

        # remove outer padding completely
        for ax_row in axes:
            for ax in ax_row:
                ax.set_position(ax.get_position())  # ensures tight packing
        plt.suptitle(f'Class-conditioned FM — epoch {epoch+1}')
        plt.tight_layout()
        plt.savefig(f'class_conditioned_sample{epoch+1}.png', dpi=300, bbox_

sampled_losses = cc_losses[::20] + [cc_losses[-1]]
iterations = list(range(0, len(cc_losses), 20)) + [len(cc_losses)-1]
plt.figure(figsize=(8, 4))
plt.plot(iterations, sampled_losses)
plt.xlabel('Iteration')
plt.ylabel('Loss')
plt.title('Class-conditioned Flow Matching Training Loss')
plt.tight_layout()
plt.savefig('training_curve_cc.png', dpi=300, bbox_inches='tight', transpare
# ===== end of code ====
```

```
100%|██████████| 938/938 [00:39<00:00, 23.56it/s]
100%|██████████| 938/938 [00:40<00:00, 23.39it/s]
100%|██████████| 938/938 [00:40<00:00, 23.39it/s]
100%|██████████| 938/938 [00:40<00:00, 23.30it/s]
100%|██████████| 938/938 [00:40<00:00, 23.28it/s]
100%|██████████| 938/938 [00:40<00:00, 23.13it/s]
100%|██████████| 938/938 [00:40<00:00, 23.25it/s]
100%|██████████| 938/938 [00:40<00:00, 23.24it/s]
100%|██████████| 938/938 [00:40<00:00, 23.30it/s]
100%|██████████| 938/938 [00:40<00:00, 23.23it/s]
```









Part 2.6: Sampling from the Class-conditioned UNet

In [ ]:
```
# Sampling from the UNet
# ===== your code here! ====
# DONE IN THE CELL ABOVE
# ===== end of code ====
```

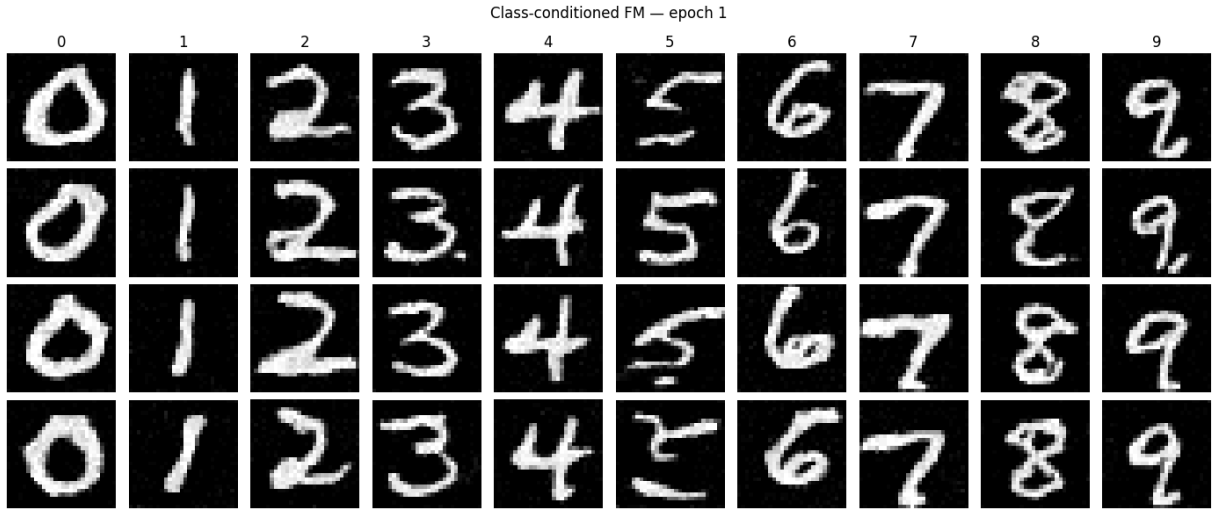
Getting rid of the annoying learning rate scheduler

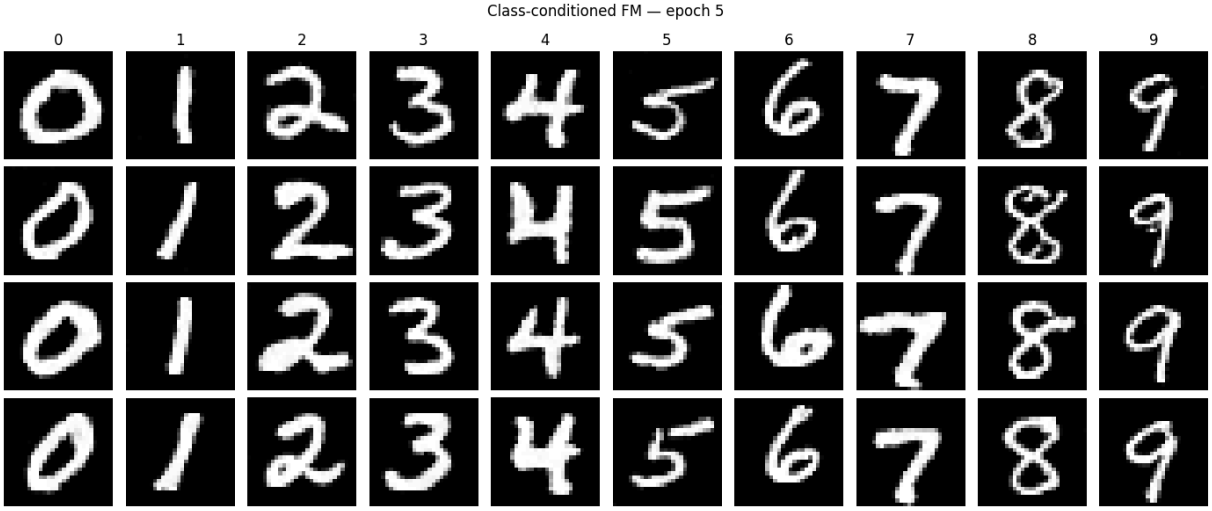
We can reduce the learning rate in order to allow a more slower and steady saturation of weights. However, reducing it implies that it will take longer for the weights to converge to the optimal value. Therefore we also have to increase the number of iterations of training.

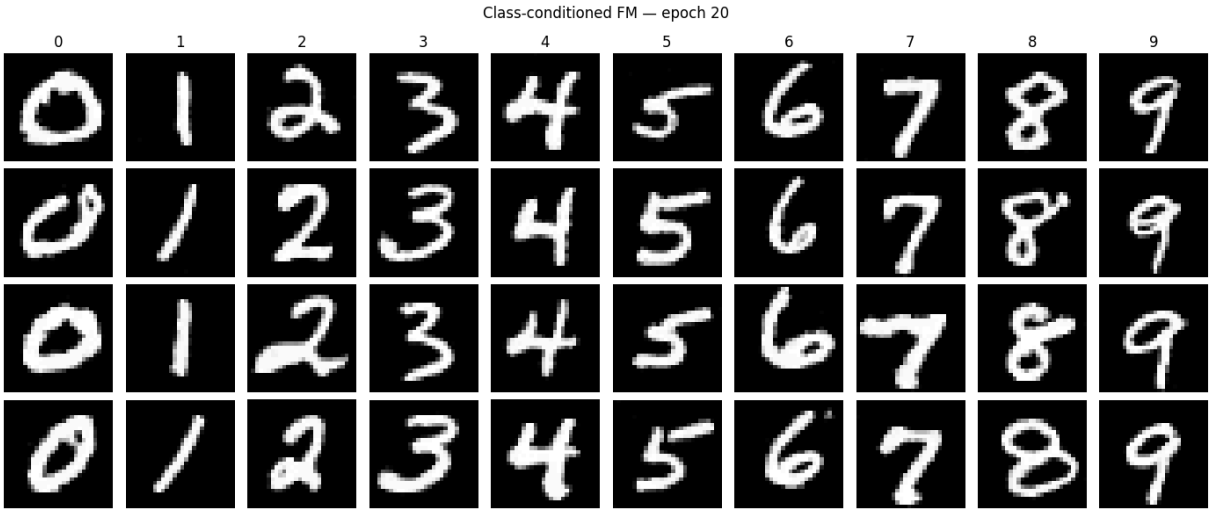
In [89]:
```
# Feel free to use code from part 1.2.1
# as they should be very similar
# ===== your code here! ====
cc_hidden_dim  = 64
cc_batch_size  = 64
cc_lr          = 1e-3
cc_num_epochs  = 20

cc_train_dataset = MNIST(root='./data', train=True,  download=True, transfor
cc_train_loader  = DataLoader(cc_train_dataset, batch_size=cc_batch_size, sh

cc_unet  = ClassConditionalUNet(in_channels=1, num_classes=10, num_hiddens=c
cc_model = ClassConditionalFM(cc_unet, num_ts=50, p_uncond=0.1).to(device)
cc_optim = torch.optim.Adam(cc_model.parameters(), lr=cc_lr)


cc_losses = []
for epoch in range(cc_num_epochs):
    cc_model.train()
    for imgs, labels in tqdm(cc_train_loader):
        imgs   = imgs.to(device)
        labels = labels.to(device)
        loss   = cc_model(imgs, labels)
        cc_optim.zero_grad()
        loss.backward()
        cc_optim.step()
        cc_losses.append(loss.item())

    if epoch + 1 in (1, 5, cc_num_epochs):
        cc_model.eval()
        # Generate 4 samples per digit (4×10 = 40 images)
        c_all = torch.arange(10, device=device).repeat_interleave(4)
        samples, _ = cc_model.sample(c=c_all, img_wh=(28, 28), seed=42)
        samples = samples.cpu().clamp(0, 1)
        fig, axes = plt.subplots(4, 10, figsize=(14, 6))
        for digit in range(10):
            for row in range(4):
                ax = axes[row, digit]
                ax.imshow(samples[digit * 4 + row].squeeze(), cmap='gray')
                ax.axis('off')
                if row == 0:
                    ax.set_title(str(digit))
        plt.subplots_adjust(wspace=0, hspace=0)
        plt.margins(0, 0)

        # remove outer padding completely
        for ax_row in axes:
            for ax in ax_row:
                ax.set_position(ax.get_position())  # ensures tight packing
        plt.suptitle(f'Class-conditioned FM — epoch {epoch+1}')
        plt.tight_layout()
        plt.savefig(f'class_conditioned_nosched_sample{epoch+1}.png', dpi=30

sampled_losses = cc_losses[::20] + [cc_losses[-1]]
iterations = list(range(0, len(cc_losses), 20)) + [len(cc_losses)-1]
```

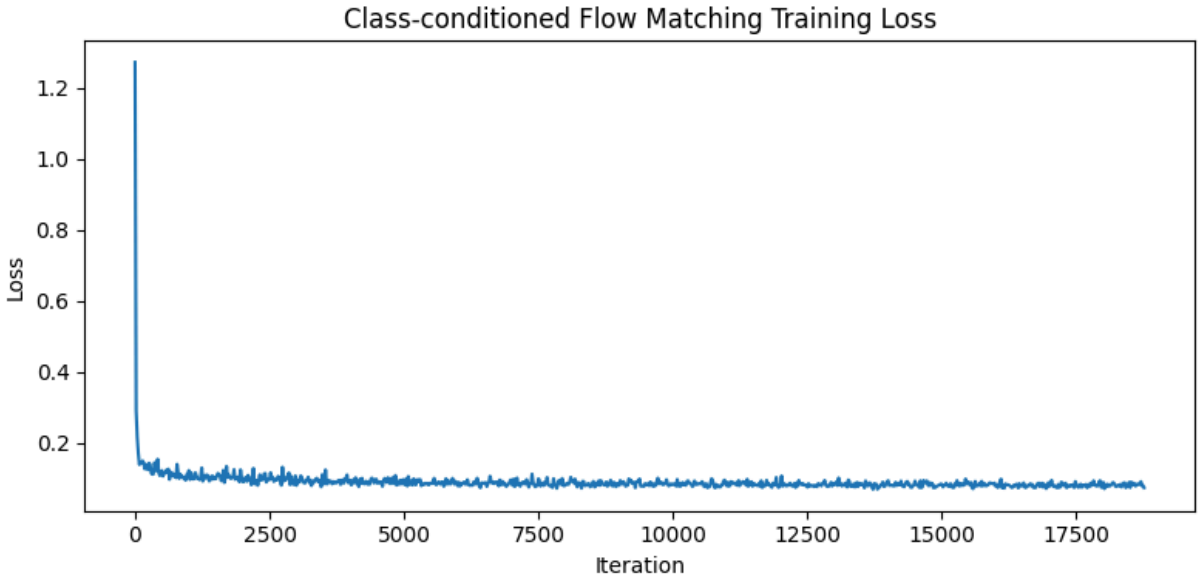
```
plt.figure(figsize=(8, 4))
plt.plot(iterations, sampled_losses)
plt.xlabel('Iteration')
plt.ylabel('Loss')
plt.title('Class-conditioned Flow Matching Training Loss')
plt.tight_layout()
plt.savefig('training_curve_cc_nosched.png', dpi=300, bbox_inches='tight', t
# ===== end of code ====
```

```
100%|██████████| 938/938 [00:41<00:00, 22.66it/s]
100%|██████████| 938/938 [00:40<00:00, 23.30it/s]
100%|██████████| 938/938 [00:40<00:00, 23.38it/s]
100%|██████████| 938/938 [00:39<00:00, 23.46it/s]
100%|██████████| 938/938 [00:40<00:00, 23.31it/s]
100%|██████████| 938/938 [00:40<00:00, 23.25it/s]
100%|██████████| 938/938 [00:40<00:00, 23.27it/s]
100%|██████████| 938/938 [00:40<00:00, 23.34it/s]
100%|██████████| 938/938 [00:40<00:00, 23.35it/s]
100%|██████████| 938/938 [00:40<00:00, 23.39it/s]
100%|██████████| 938/938 [00:40<00:00, 23.28it/s]
100%|██████████| 938/938 [00:40<00:00, 23.38it/s]
100%|██████████| 938/938 [00:40<00:00, 23.37it/s]
100%|██████████| 938/938 [00:40<00:00, 23.32it/s]
100%|██████████| 938/938 [00:40<00:00, 23.26it/s]
100%|██████████| 938/938 [00:40<00:00, 23.35it/s]
100%|██████████| 938/938 [00:40<00:00, 23.33it/s]
100%|██████████| 938/938 [00:40<00:00, 23.26it/s]
100%|██████████| 938/938 [00:40<00:00, 23.39it/s]
100%|██████████| 938/938 [00:40<00:00, 23.36it/s]
```




Class-conditioned FM — epoch 5
0
1
2
3
4
5
6
7
8
9


Class-conditioned FM — epoch 20
0
1
2
3
4
5
6
7
8
9


Class-conditioned Flow Matching Training Loss
Loss
1.2
1.0
0.8
0.6
0.4
0.2
0
2500
5000
7500
10000
12500
15000
17500
Iteration